

Software Design Document

MSBON Fraud-Sensitive Transcript Verification System

Version 1.0 | Team Nexus | University of Southern Mississippi

Revision History

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
 - 1.5 Document Overview
2. System Overview
 - 2.1 Background and Problem Statement
 - 2.2 System Purpose
 - 2.3 Stakeholders
 - 2.4 System Context Diagram (C4 Level 1)
 - 2.5 Assumptions and Constraints
3. Architectural Design
 - 3.1 Architectural Style
 - 3.2 Container Diagram (C4 Level 2)
 - 3.3 Backend Component Diagram (C4 Level 3)
 - 3.4 Frontend Component Hierarchy
 - 3.5 Deployment Diagram (PoC)
4. Data Design
 - 4.1 Entity-Relationship Diagram
 - 4.2 Database Schema
 - 4.3 Transcript State Machine
 - 4.4 Data Flow Diagram
5. Component Design
 - 5.1 Class Diagram — Domain Models
 - 5.2 Class Diagram — Service Layer
 - 5.3 Class Diagram — Rule Engine
 - 5.4 Flagging Rules Reference
 - 5.5 Class Diagram — Infrastructure Layer
6. Interface Design
 - 6.1 REST API Overview
 - 6.2 Endpoint Contracts
 - 6.3 Sequence Diagram — Full Transcript Verification Flow
 - 6.4 Sequence Diagram — Staff Review Decision
 - 6.5 LLM Prompt Design (Google Gemini)
7. Security Design
 - 7.1 Threat Model (STRIDE)
 - 7.2 File Upload Security Controls
 - 7.3 Authentication and Authorization
 - 7.4 CORS Configuration
 - 7.5 Observability and Audit Hardening
 - 7.6 Data Privacy Controls
8. Error Handling & Resilience
 - 8.1 Error Code Reference
 - 8.2 LLM Retry and Resilience Strategy

- 8.3 Global Exception Handler
- 9. Testing Strategy
 - 9.1 Test Pyramid
 - 9.2 Unit Test Coverage Targets
 - 9.3 Integration Test Scenarios
 - 9.4 Synthetic Test Transcript Inventory
 - 9.5 Acceptance Criteria (from Scope Document)
- 10. Non-Functional Requirements
- 11. Appendix
 - 11.1 Project Directory Structure
 - 11.2 Environment Variables
 - 11.3 Backend Dependency Manifest
 - 11.4 Frontend Dependency Manifest
 - 11.5 Flagging Rules Seed Data

Software Design Document

MSBON Fraud-Sensitive Transcript Verification System

Version 1.0 | Team Nexus | University of Southern Mississippi

Revision History

Version	Date	Author	Description
0.1	2026-03-28	Team Nexus	Initial draft
1.0	2026-03-28	Team Nexus	Baseline SDD — PoC scope

Table of Contents

- 1. [Introduction](#)
- 2. [System Overview](#)
- 3. [Architectural Design](#)
- 4. [Data Design](#)
- 5. [Component Design](#)
- 6. [Interface Design](#)
- 7. [Security Design](#)
- 8. [Error Handling & Resilience](#)
- 9. [Testing Strategy](#)
- 10. [Non-Functional Requirements](#)
- 11. [Appendix](#)

1. Introduction

1.1 Purpose

This Software Design Document (SDD) provides the complete technical specification for the MSBON Fraud-Sensitive Transcript Verification System — a Proof of Concept (PoC) developed for the Mississippi State Board of Nursing (MSBON) by Team Nexus at The University of Southern Mississippi. It describes the system’s architecture, data models, component interactions, API contracts, security posture, and testing strategy, and serves as the authoritative reference for development and stakeholder review.

1.2 Scope

The system automates the extraction and rule-based verification of nursing school transcripts submitted to MSBON for licensure applications. It flags potential anomalies — missing graduation confirmation, accreditation mismatches, course deficiencies, fraud indicators, and formatting inconsistencies — for human staff review. All staff decisions and system actions are captured in tamper-evident audit logs. **No automated licensure decisions are ever made by this system.**

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
MSBON / MSBN	Mississippi State Board of Nursing
PoC	Proof of Concept
OCR	Optical Character Recognition
LLM	Large Language Model
SDD	Software Design Document
ACEN	Accreditation Commission for Education in Nursing
CCNE	Commission on Collegiate Nursing Education
API	Application Programming Interface
REST	Representational State Transfer
ORM	Object-Relational Mapper
SPA	Single Page Application
SOP	Standard Operating Procedure
ER	Entity-Relationship
UML	Unified Modeling Language
FERPA	Family Educational Rights and Privacy Act
ADN	Associate Degree in Nursing
BSN	Bachelor of Science in Nursing
MSN	Master of Science in Nursing
ASGI	Asynchronous Server Gateway Interface

1.4 References

- MSBON PoC Scope Document (January 7, 2025)
- MSBON Project Proposal, Team Nexus (February 27, 2026)
- Operation Nightingale Federal Investigation
- ACEN and CCNE Accreditation Standards
- FastAPI Official Documentation
- Google Gemini API Documentation
- OWASP Top 10 Security Guidelines (2021)
- PEP 8 — Python Style Guide

1.5 Document Overview

Section 2 provides a system overview and stakeholder context. Section 3 covers layered architecture with C4 container and component diagrams. Section 4 details the data design including ER diagram, database schema, and state machine. Section 5 defines class diagrams for every layer. Section 6 specifies REST API contracts and sequence diagrams. Section 7 addresses security. Section 8 covers error handling. Section 9 defines the testing strategy. Section 10 lists non-functional requirements. Section 11 provides the directory structure, environment variables, and dependency manifest.

2. System Overview

2.1 Background and Problem Statement

The Mississippi State Board of Nursing currently reviews nursing school transcripts manually to confirm that applicants have completed an approved nursing education program. This process is time-consuming, inconsistent across institutions, and vulnerable to human error. Federal investigations such as Operation Nightingale demonstrated that bad actors can exploit gaps in manual review processes to fraudulently obtain nursing licenses — a direct threat to public safety.

2.2 System Purpose

The MSBON Transcript Verification System is an AI-assisted tool that:

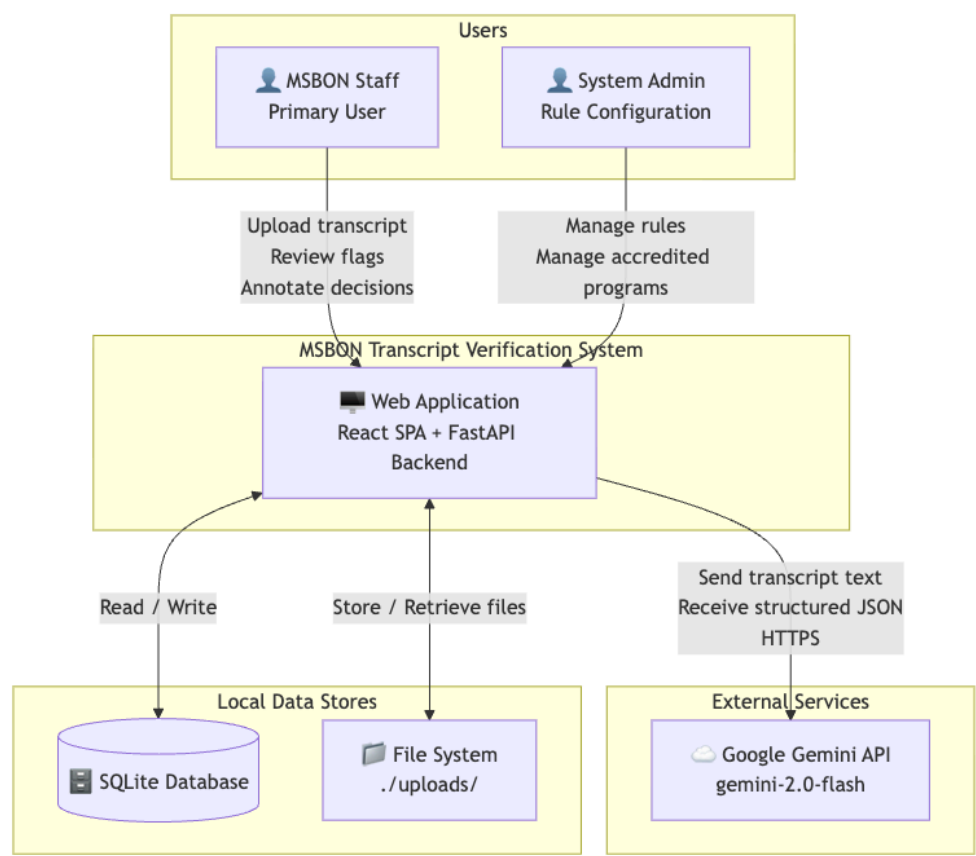
1. Ingests nursing transcripts (PDF or scanned image)
2. Extracts structured data using OCR and a Large Language Model (Google Gemini)
3. Applies transparent, rule-based verification logic against MSBON-defined licensure requirements
4. Flags anomalies with full explanations for staff review
5. Provides a staff workflow for confirming, overriding, and annotating findings
6. Generates audit-ready logs documenting every action

All AI outputs are **advisory only**. Human staff retain full decision-making authority.

2.3 Stakeholders

Role	Person(s)	Interaction with System
Business Decision Maker	Shantannar Montgomery	Product direction, acceptance
Data Validators	Dr. Burks, Dr. Smith, Talisha Greenlaw	Rule accuracy validation
Primary Users	MSBON Staff	Upload, review, annotate
Development Team	Team Nexus (Sandesh, Nishit, Kiran, Chance, Suvi)	Build and test

2.4 System Context Diagram (C4 Level 1)



Graph 1

2.5 Assumptions and Constraints

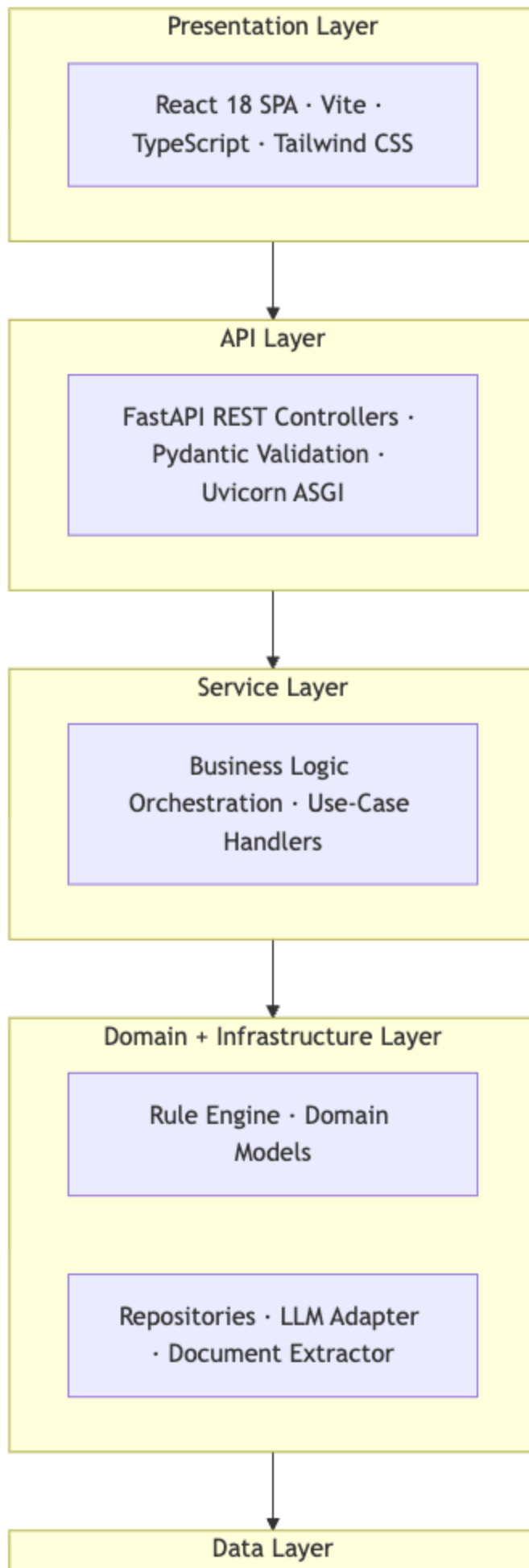
Constraint	Detail
Scope	PoC only — not for production deployment or public access

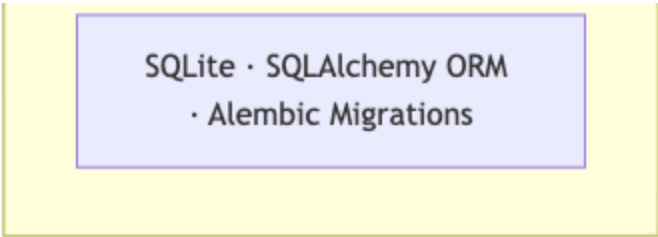
Constraint	Detail
Transcript origin	Only domestic (U.S.) transcripts in scope
Data	Only synthetic or de-identified sample transcripts during development
Storage	No permanent retention of real transcript content
Network	Operates on local or private network
AI authority	All AI outputs advisory — no automated licensure decisions
Runtime	Python 3.11+, Node.js 18+

3. Architectural Design

3.1 Architectural Style

The system follows a **Layered Architecture** pattern with strict separation of concerns across five layers.



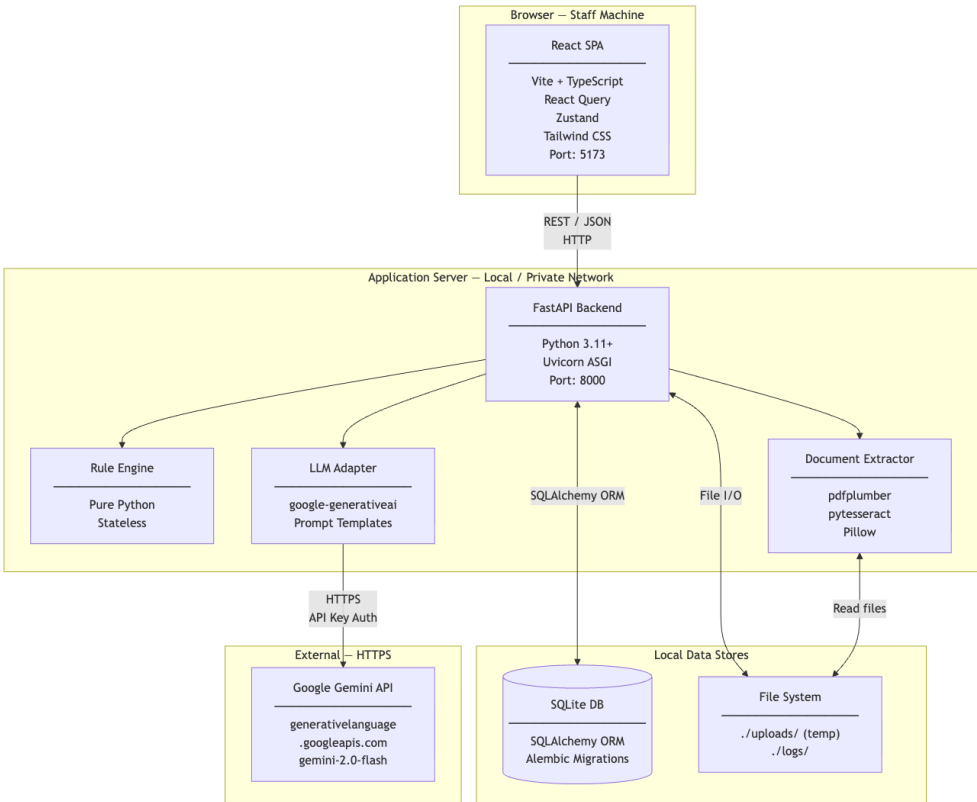


Graph 2

Why layered architecture:

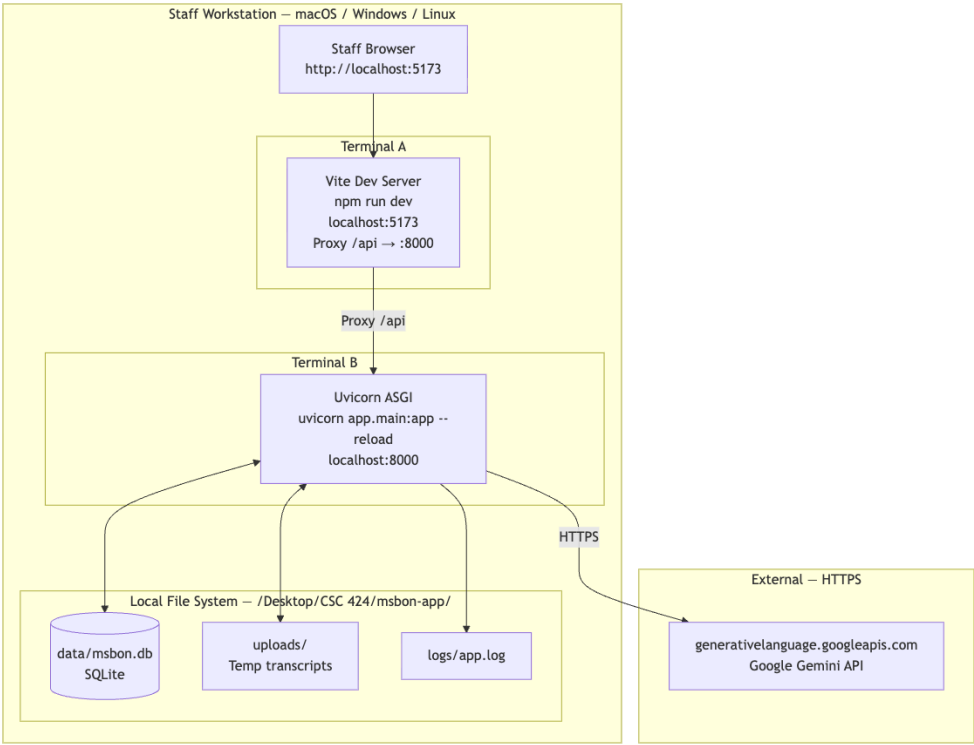
Benefit	How it applies
Testability	Each layer tested independently with mocks at boundaries
Auditability	All business logic isolated — every flag traceable to a rule class
Maintainability	Rule engine evolves without touching API or UI
Explainability	Transparent rule logic required by scope; no black-box processing

3.2 Container Diagram (C4 Level 2)



Graph 3

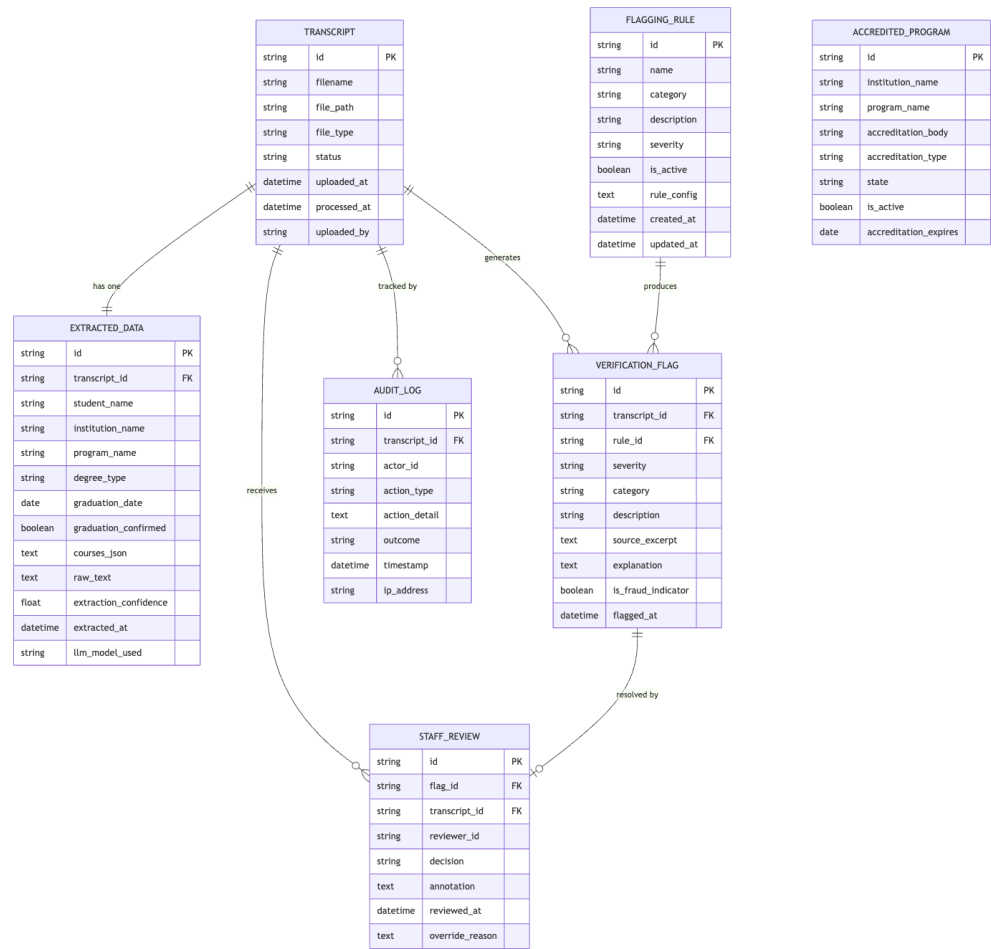
3.5 Deployment Diagram (PoC)



Graph 6

4. Data Design

4.1 Entity-Relationship Diagram



Er Diagram 1

4.2 Database Schema

Table: transcripts

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4
filename	VARCHAR(255)	NOT NULL	Original filename (sanitized for display only)
file_path	VARCHAR(512)	NOT NULL	Relative path under ./uploads/
file_type	VARCHAR(10)	NOT NULL, CHECK IN ('pdf', 'image')	File category
status	VARCHAR(20)	NOT NULL	See §4.3 state machine
uploaded_at	DATETIME	NOT NULL	UTC timestamp

Column	Type	Constraints	Description
processed_at	DATETIME	NULLABLE	UTC — set when verification completes
uploaded_by	VARCHAR(100)	NOT NULL	Staff identifier from X-Staff-ID header

Table: extracted_data

Column	Type	Constraints	Description
id	VARCHAR(36)	PK	UUID v4
transcript_id	VARCHAR(36)	FK → transcripts.id, UNIQUE	One-to-one with transcript
student_name	VARCHAR(255)	NULLABLE	Full name as extracted
institution_name	VARCHAR(255)	NULLABLE	School name as extracted
program_name	VARCHAR(255)	NULLABLE	Nursing program name
degree_type	VARCHAR(20)	NULLABLE	ADN / BSN / MSN / DNP / LPN / OTHER
graduation_date	DATE	NULLABLE	ISO 8601
graduation_confirmed	BOOLEAN	NOT NULL DEFAULT FALSE	True only when explicit conferral statement present
courses_json	TEXT	NULLABLE	JSON array of course objects
raw_text	TEXT	NULLABLE	Full text from OCR / PDF extraction
extraction_confidence	REAL	NULLABLE	0.0–1.0 model confidence
extracted_at	DATETIME	NOT NULL	UTC
llm_model_used	VARCHAR(100)	NOT NULL	e.g., gemini-2.0-flash

Table: verification_flags

Column	Type	Constraints	Description
id	VARCHAR(36)	PK	UUID v4
transcript_id	VARCHAR(36)	FK → transcripts.id	
rule_id	VARCHAR(36)	FK → flagging_rules.id	Rule that produced this flag
severity	VARCHAR(10)	NOT NULL	HIGH / MEDIUM / LOW

Column	Type	Constraints	Description
category	VARCHAR(20)	NOT NULL	GRADUATION / ACCREDITATION / COURSE / FRAUD / FORMAT
description	VARCHAR(500)	NOT NULL	One-line human- readable description
source_excerpt	TEXT	NULLABLE	Verbatim text from transcript that triggered flag
explanation	TEXT	NOT NULL	Full rule rationale with rule ID
is_fraud_indicator	BOOLEAN	NOT NULL	
flagged_at	DATETIME	NOT NULL	UTC

Table: staff_reviews

Column	Type	Constraints	Description
id	VARCHAR(36)	PK	UUID v4
flag_id	VARCHAR(36)	FK → verification_flags.id, UNIQUE	One review per flag
transcript_id	VARCHAR(36)	FK → transcripts.id	Denormalized for query convenience
reviewer_id	VARCHAR(100)	NOT NULL	Staff identifier
decision	VARCHAR(20)	NOT NULL	CONFIRMED / OVERRIDDEN / NEEDS_MORE_INFO
annotation	TEXT	NULLABLE	Staff notes
reviewed_at	DATETIME	NOT NULL	UTC
override_reason	TEXT	NULLABLE	Required when decision = OVERRIDDEN

Table: audit_logs

Column	Type	Constraints	Description
id	VARCHAR(36)	PK	UUID v4
transcript_id	VARCHAR(36)	FK NULLABLE	Null for system-level events
actor_id	VARCHAR(100)	NOT NULL	Staff or system identifier

Column	Type	Constraints	Description
action_type	VARCHAR(50)	NOT NULL	UPLOAD / EXTRACT / VERIFY / REVIEW_FLAG / OVERRIDE_FLAG / EXPORT / VIEW
action_detail	TEXT	NOT NULL	JSON blob with full context
outcome	VARCHAR(20)	NOT NULL	SUCCESS / FAILURE / PARTIAL
timestamp	DATETIME	NOT NULL	UTC
ip_address	VARCHAR(45)	NULLABLE	IPv4 or IPv6

Immutability constraint: No UPDATE or DELETE operations are permitted on audit_logs. Enforced at the AuditRepository layer — save() is the only allowed write operation.

Table: flagging_rules

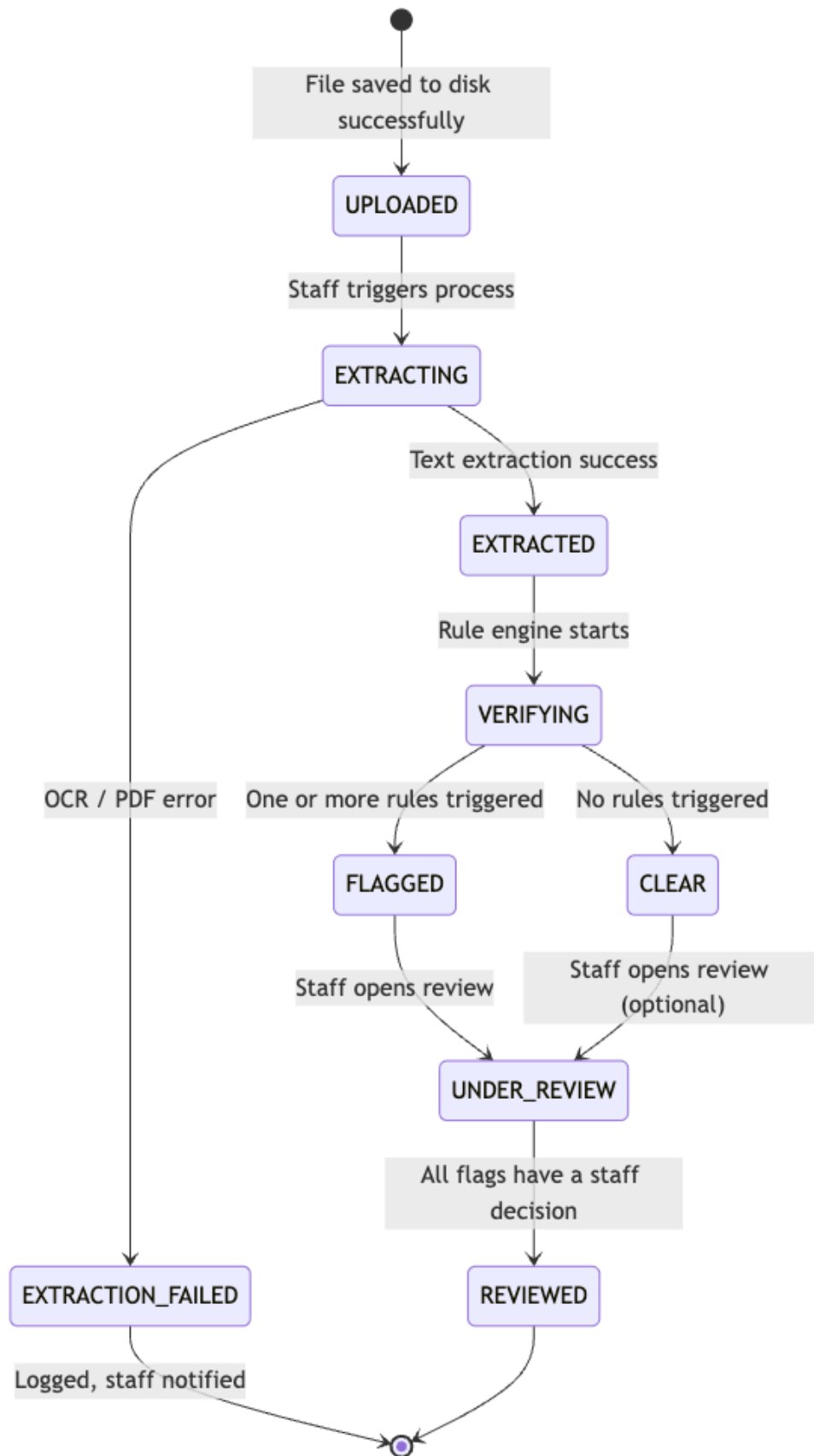
Column	Type	Constraints	Description
id	VARCHAR(36)	PK	UUID v4
name	VARCHAR(100)	NOT NULL, UNIQUE	Rule short name (e.g., GRAD-001)
category	VARCHAR(20)	NOT NULL	GRADUATION / ACCREDITATION / COURSE / FRAUD / FORMAT
description	TEXT	NOT NULL	Full description of what this rule checks
severity	VARCHAR(10)	NOT NULL	HIGH / MEDIUM / LOW
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	Disabled rules are skipped by RuleEngine
rule_config	TEXT	NULLABLE	JSON for configurable thresholds (e.g., min program months)
created_at	DATETIME	NOT NULL	UTC
updated_at	DATETIME	NOT NULL	UTC

Table: accredited_programs

Column	Type	Constraints	Description
id	VARCHAR(36)	PK	UUID v4
institution_name	VARCHAR(255)	NOT NULL	Full institution name

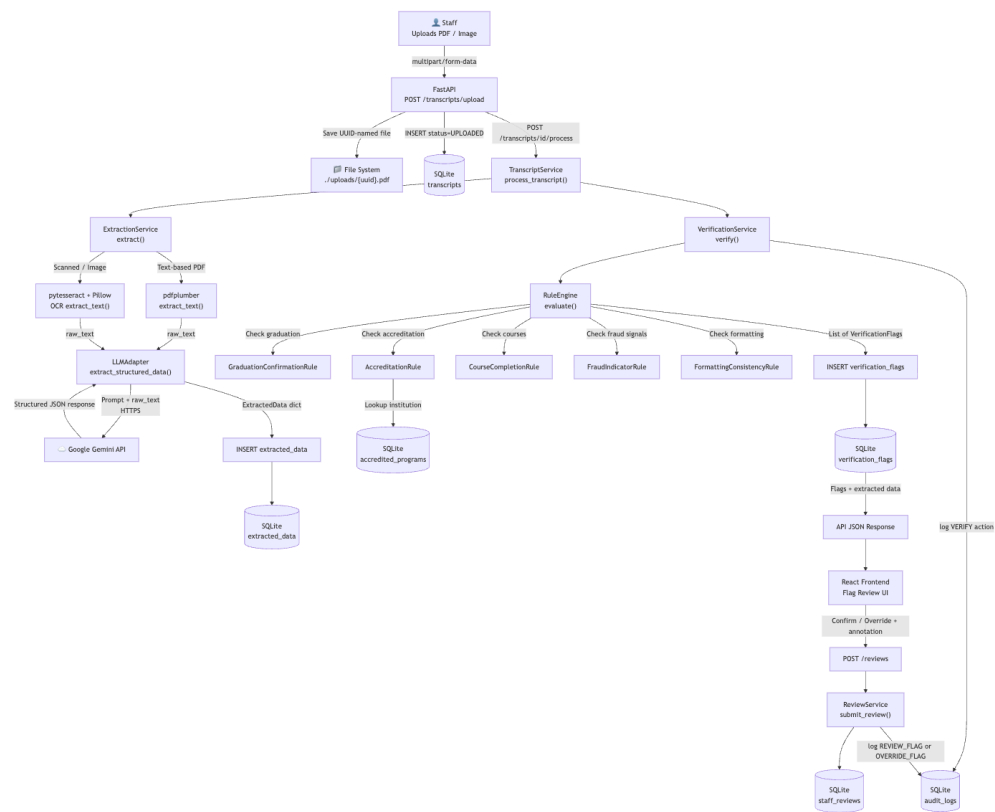
Column	Type	Constraints	Description
program_name	VARCHAR(255)	NOT NULL	Nursing program name
accreditation_body	VARCHAR(20)	NOT NULL	ACEN / CCNE / STATE / OTHER
accreditation_type	VARCHAR(50)	NOT NULL	
state	VARCHAR(2)	NOT NULL	Two-letter state code
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	
accreditation_expires	DATE	NULLABLE	

4.3 Transcript State Machine



State Diagram 1

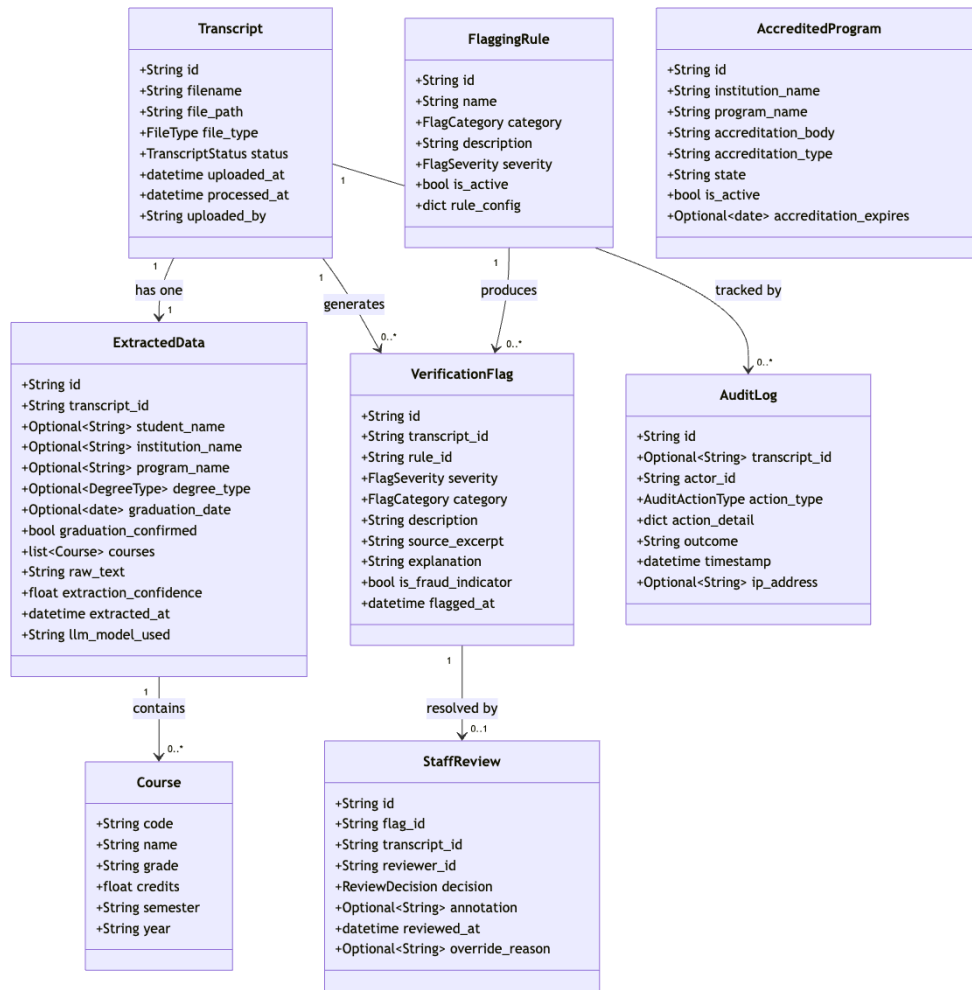
4.4 Data Flow Diagram



Flowchart 1

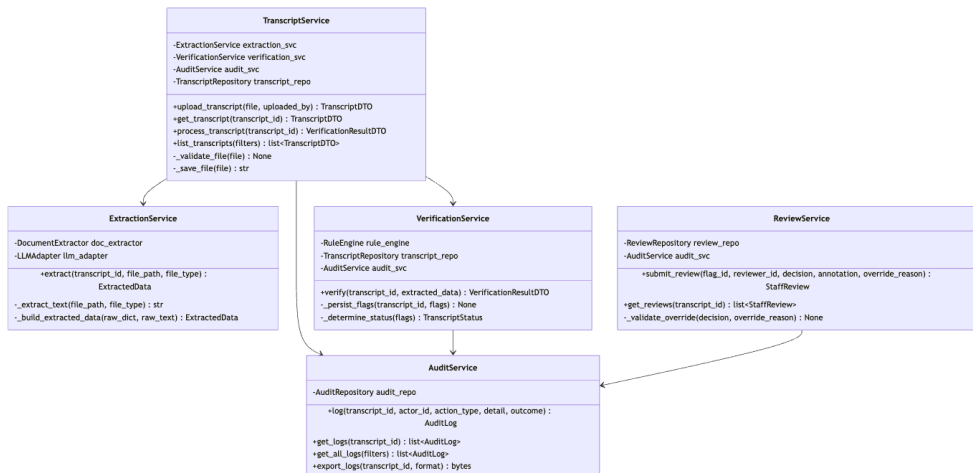
5. Component Design

5.1 Class Diagram — Domain Models



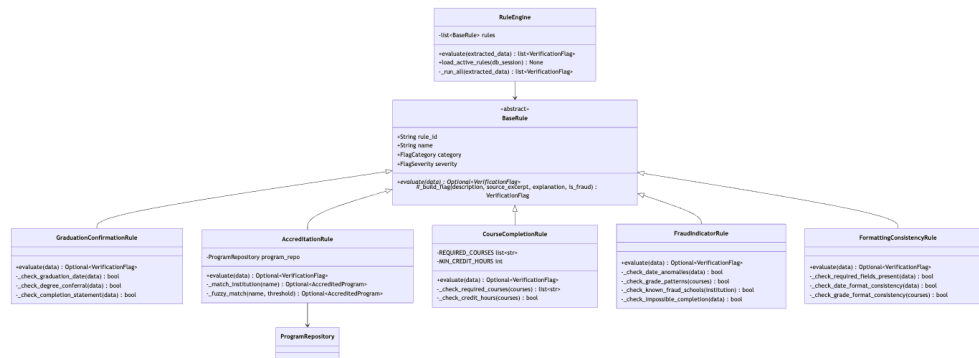
Class Diagram 1

5.2 Class Diagram — Service Layer



Class Diagram 2

5.3 Class Diagram — Rule Engine



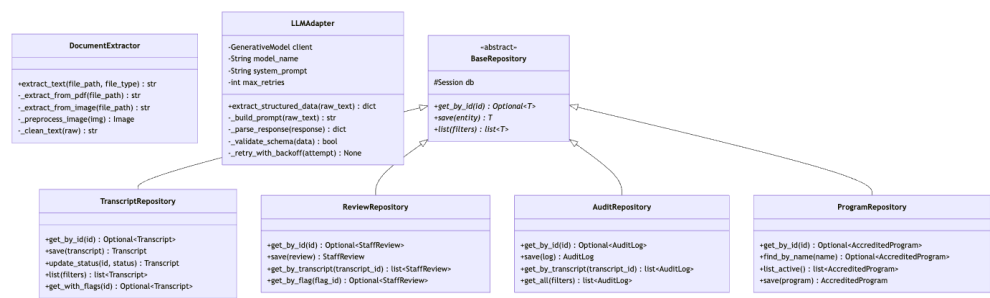
Class Diagram 3

5.4 Flagging Rules Reference

Rule ID	Name	Category	Severity	Description
GRAD-001	Missing graduation confirmation	GRADUATION	HIGH	No explicit degree conferral statement found
GRAD-002	Graduation date absent	GRADUATION	MEDIUM	Graduation date field not present
ACCR-001	Institution not in MS accredited list	ACCREDITATION	HIGH	School not found in accredited_programs table
ACCR-002	Accreditation body not recognized	ACCREDITATION	MEDIUM	Body is not ACEN or CCNE
COUR-001	Required nursing courses not present	COURSE	HIGH	One or more mandatory courses missing from course list
COUR-002	Insufficient credit hours	COURSE	MEDIUM	Total nursing credits below minimum threshold
FRAU-001	Program completion time below minimum	FRAUD	HIGH	Duration from enrollment to graduation is physically impossible
FRAU-002	Suspicious grade distribution	FRAUD	MEDIUM	Abnormal pattern suggesting grade fabrication
FRAU-003	Institution on fraud watch list	FRAUD	HIGH	School name matches known fraudulent program
FORM-001	Required transcript	FORMAT	LOW	Key fields (name, school, program)

Rule ID	Name	Category	Severity	Description
	fields missing			could not be extracted
FORM-002	Inconsistent date formatting	FORMAT	LOW	Dates appear in multiple conflicting formats
FORM-003	Inconsistent grade formatting	FORMAT	LOW	Grade notation changes mid-transcript

5.5 Class Diagram — Infrastructure Layer



Class Diagram 4

AuditRepository immutability note: The AuditRepository deliberately omits update() and delete() methods. Any attempt to modify audit log records raises OperationNotPermittedError at the repository boundary.

6. Interface Design

6.1 REST API Overview

Base URL: <http://localhost:8000/api/v1>

Method	Endpoint	Description
POST	/transcripts/upload	Upload a transcript file
POST	/transcripts/{id}/process	Trigger extraction + verification
GET	/transcripts/{id}	Get transcript with extracted data + flags
GET	/transcripts	List all transcripts (paginated, filterable)
POST	/reviews	Submit a staff review decision
GET	/reviews/{transcript_id}	Get all reviews for a transcript
GET	/audit-logs	Get all audit logs (filterable)

Method	Endpoint	Description
GET	/audit-logs/{transcript_id}	Get audit trail for one transcript
GET	/audit-logs/{transcript_id}/export	Export audit trail (CSV or JSON)
GET	/programs	List accredited nursing programs
POST	/programs	Add an accredited program (admin)
GET	/health	System health check

Standard Request Headers:

Header	Required On	Value
X-Staff-ID	All write operations	Staff identifier string
X-Staff-Role	Admin-only endpoints	admin
Content-Type	JSON body requests	application/json

Standard Error Response Shape:

```
{
  "error": {
    "code": "ERROR_CODE",
    "message": "Human-readable message",
    "detail": "Optional technical detail",
    "transcript_id": "optional-uuid",
    "timestamp": "2026-03-28T14:00:00Z"
  }
}
```

6.2 Endpoint Contracts

POST /transcripts/upload

Upload a transcript file for processing.

Request: multipart/form-data

Field	Type	Required	Description
file	binary	Yes	PDF, PNG, JPEG, or TIFF — max 10MB
uploaded_by	string	Yes	Staff identifier

Response 201 Created:

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "filename": "transcript_jane_doe.pdf",
  "status": "UPLOADED",
```

```
"uploaded_at": "2026-03-28T14:00:00Z",
"uploaded_by": "staff_dr_burks"
}
```

Error responses:

Status	Code	Condition
400	INVALID_FILE_TYPE	File is not PDF / PNG / JPEG / TIFF
413	FILE_T00_LARGE	File exceeds 10MB

POST /transcripts/{id}/process

Trigger the extraction and verification pipeline.

Response 202 Accepted:

```
{
  "transcript_id": "3fa85f64-...",
  "status": "EXTRACTING",
  "message": "Processing pipeline started"
}
```

GET /transcripts/{id}

Retrieve a transcript with extracted data and all flags.

Response 200 OK:

```
{
  "id": "3fa85f64-...",
  "filename": "transcript_jane_doe.pdf",
  "status": "FLAGGED",
  "uploaded_at": "2026-03-28T14:00:00Z",
  "processed_at": "2026-03-28T14:01:32Z",
  "extracted_data": {
    "student_name": "Jane Doe",
    "institution_name": "University of Southern Mississippi",
    "program_name": "Bachelor of Science in Nursing",
    "degree_type": "BSN",
    "graduation_date": "2024-05-15",
    "graduation_confirmed": true,
    "courses": [
      {
        "code": "NUR 301",
        "name": "Fundamentals of Nursing",
        "grade": "A",
        "credits": 3.0,
        "semester": "Fall",
        "year": "2022"
      }
    ]
  }
}
```

```
    }
  ],
  "extraction_confidence": 0.94,
  "extracted_at": "2026-03-28T14:01:05Z"
},
"flags": [
  {
    "id": "flag-uuid",
    "severity": "HIGH",
    "category": "FRAUD",
    "description": "Program completion time below minimum threshold",
    "source_excerpt": "Enrollment: January 2024 | Graduation: May 2024",
    "explanation": "Rule FRAU-001: BSN program requires minimum 24 months. Detected completion in approximately 4 months.",
    "is_fraud_indicator": true,
    "flagged_at": "2026-03-28T14:01:30Z",
    "review": null
  }
]
}
```

POST /reviews

Submit a staff review decision on a specific flag.

Request body:

```
{
  "flag_id": "flag-uuid",
  "transcript_id": "transcript-uuid",
  "reviewer_id": "staff_dr_burks",
  "decision": "OVERRIDDEN",
  "annotation": "Applicant was in an accelerated BSN bridge program.",
  "override_reason": "Accelerated program documentation verified separately."
}
```

Decision values:

Value	Meaning	override_reason required?
CONFIRMED	Staff agrees with the flag	No
OVERRIDDEN	Staff dismisses the flag	Yes
NEEDS_MORE_INFO	Additional documentation required	No

Response 201 Created:

```
{
  "id": "review-uuid",
  "flag_id": "flag-uuid",
  "decision": "OVERRIDDEN",
  "annotation": "Applicant was in an accelerated BSN bridge program.",
  "reviewed_at": "2026-03-28T14:30:00Z"
}
```

Error responses:

Status	Code	Condition
404	TRANSCRIPT_NOT_FOUND	Transcript ID doesn't exist
409	TRANSCRIPT_NOT_PROCESSED	Transcript not yet verified
409	FLAG_ALREADY_REVIEWED	Flag already has a staff review
422	OVERRIDE_REASON_REQUIRED	Decision is OVERRIDDEN but reason is missing

GET /audit-logs/{transcript_id}/export

Query params:

Param	Values	Default
format	csv json	json

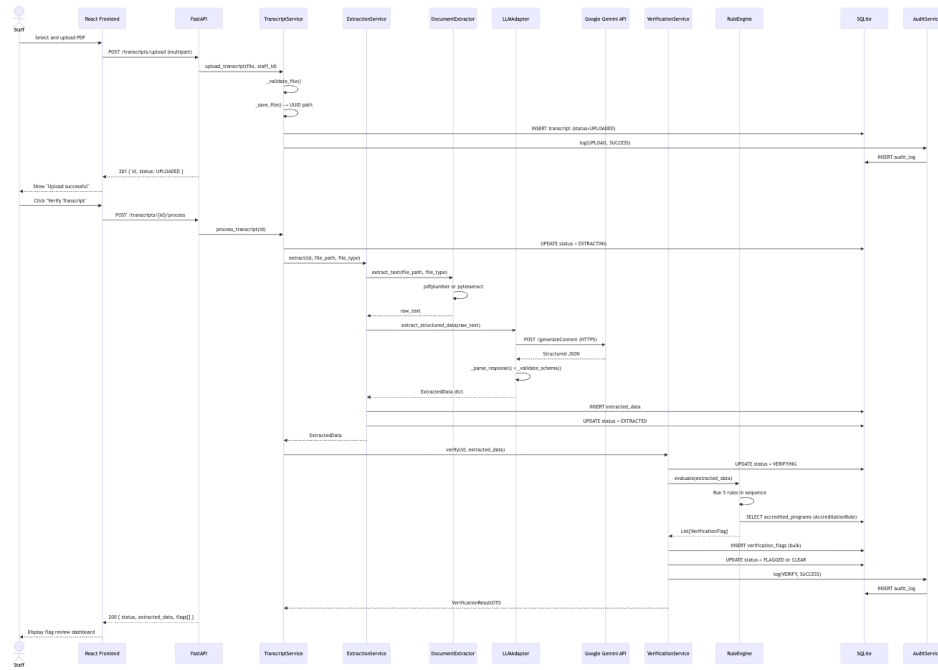
Response: Binary file download — Content-Disposition: attachment; filename="audit_{transcript_id}.{format}"

GET /health

Response 200 OK:

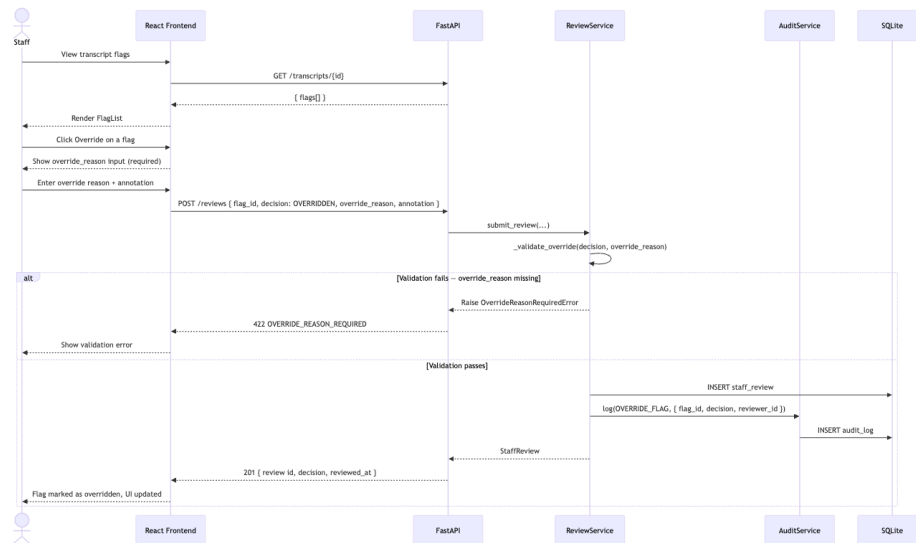
```
{
  "status": "healthy",
  "database": "connected",
  "llm_api": "reachable",
  "version": "1.0.0-poc",
  "timestamp": "2026-03-28T14:00:00Z"
}
```


6.3 Sequence Diagram — Full Transcript Verification Flow



Sequence Diagram 1

6.4 Sequence Diagram — Staff Review Decision



Sequence Diagram 2

6.5 LLM Prompt Design (Google Gemini)

System Instruction:

You are a transcript verification assistant for the Mississippi State Board of Nursing (MSBON).
Your only task is to extract structured information from nursing school transcripts.
Do not infer, guess, or fabricate any information.
If a field cannot be determined from the document, return null for

that field.

Return ONLY a valid JSON object – no explanation, no markdown, no code fences.

User Prompt Template:

Extract the following information from this nursing school transcript and return it as JSON.

Required output schema:

```
{
  "student_name":      string | null,
  "institution_name":   string | null,
  "program_name":      string | null,
  "degree_type":       "ADN" | "BSN" | "MSN" | "DNP" | "LPN" |
  "OTHER" | null,
  "graduation_date":   "YYYY-MM-DD" | null,
  "graduation_confirmed": boolean,
  "enrollment_date":   "YYYY-MM-DD" | null,
  "courses": [
    {
      "code":      string | null,
      "name":      string,
      "grade":     string,
      "credits":   number | null,
      "semester":  string | null,
      "year":      string | null
    }
  ],
  "extraction_notes": string | null
}
```

Rules:

- graduation_confirmed = true ONLY when an explicit degree conferral or graduation statement is present
- Return dates strictly in ISO 8601 format (YYYY-MM-DD)
- Do not populate courses array with non-nursing courses if easily distinguishable

TRANSCRIPT TEXT:

```
---
{raw_text}
---
```

7. Security Design

Phase A note: Authentication and authorization in this section reflect the JWT-based implementation that landed in Phase A of the remediation plan (backend/app/auth/permissions.py, backend/app/api/v1/auth.py,

backend/app/api/v1/dependencies.py). The earlier X-Staff-ID / X-Staff-Role header pattern is removed and **must not** be reintroduced.

7.1 Threat Model (STRIDE)

Category	Threat	Risk Level	Mitigation
Spoofing	Impersonating MSBON staff	HIGH	Stateless JWT (HS256) with server-side sec client-supplied identity headers ignored
Tampering	Modifying audit logs after creation	HIGH	AuditRepository.update() and delete OperationNotPermittedError; write-on layer
Repudiation	Staff denying a review action	HIGH	Every review stores reviewer_id (sourcec sub claim, not request body), decision, an override_reason, reviewed_at, and a c
Info Disclosure	Transcript PII leaked via API response	HIGH	File paths never exposed; no PII in log mes stored outside web root; audit action_det transcript_id only
Info Disclosure	Transcript content sent externally via LLM	MEDIUM	Only synthetic/de-identified data in PoC; al over HTTPS to a single allow-listed host
Info Disclosure	API key leak via logs / responses	HIGH	GEMINI_API_KEY and JWT_SECRET read fr never logged or returned; redacted in any st scrubber
Denial of Service	Upload request flooding	LOW	10 MB hard size limit at FastAPI; rate limit Phase B
Elevation of Privilege	Non-admin accessing program / rule management	MEDIUM	require_permission(Permission.MANA dependency on every admin route; permissi exclusively from JWT role claim
Elevation of Privilege	Identity impersonation via request body	MEDIUM	reviewer_id is never read from the requ always taken from the verified JWT sub cl

7.2 File Upload Security Controls

Control	Implementation
MIME type validation	python-magic reads file magic bytes — not file extension
Cross-validation	Detected MIME must match the claimed extension; mismatch → 422
Filename sanitization	Files saved as {uuid}.{ext} — original filename used for display only

Control	Implementation
Path traversal prevention	Upload directory is a fixed absolute path; filenames never concatenated raw
Size limit	Enforced at FastAPI middleware before file is written to disk
Storage isolation	./uploads/ is outside the static file serving root
Retention	Files deleted after FILE_RETENTION_HOURS (default 24 h) via background task

7.3 Authentication and Authorization

7.3.1 JWT Issuance and Verification

Authentication is stateless and JWT-based, implemented with python-jose and HMAC-SHA256.

Element	Value
Algorithm	HS256 (configurable via JWT_ALGORITHM)
Secret	JWT_SECRET env var, minimum 32 bytes; system refuses to start otherwise
Token lifetime	480 minutes (8-hour shift), configurable via JWT_EXPIRE_MINUTES
Payload	{"sub": "<staff_id>", "role": "admin\ reviewer\ viewer", "exp": <unix-ts>}
Issued by	POST /api/v1/auth/login
Verified by	verify_token dependency in app/api/v1/dependencies.py — applied to every protected route

7.3.2 Role-Based Access Control

```
# app/auth/permissions.py
ROLE_PERMISSIONS = {
    "admin":    {MANAGE_PROGRAMS, SUBMIT_REVIEW, VIEW_TRANSCRIPTS,
                  EXPORT_AUDIT, UPLOAD_TRANSCRIPT},
    "reviewer": {SUBMIT_REVIEW, VIEW_TRANSCRIPTS},
    "viewer":   {VIEW_TRANSCRIPTS},
}

def require_permission(p: Permission) -> Callable:
    async def _check(token: dict = Depends(verify_token)) -> None:
        if p not in ROLE_PERMISSIONS.get(token.get("role", ""),
                                           set()):
            set():
```

```

        raise HTTPException(status_code=403, detail={"error":
        {...}})
    return _check

```

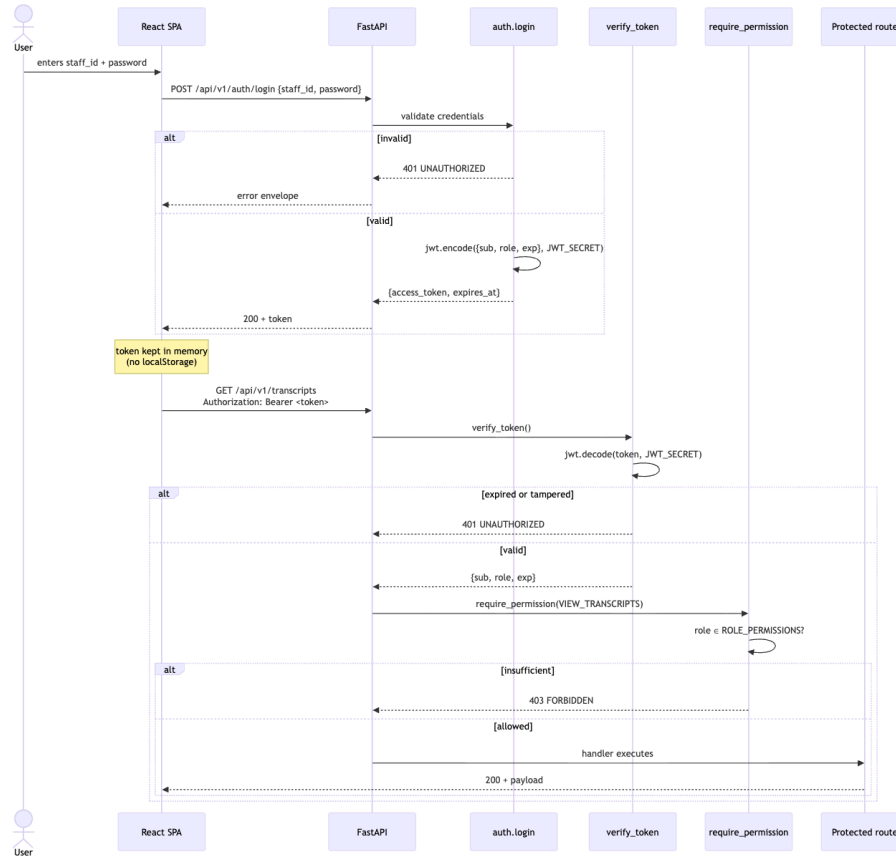
Routes declare their required permission as a FastAPI dependency, e.g.:

```

@router.post("/transcripts", dependencies=
    [Depends(require_permission(Permission.UPLOAD_TRANSCRIPT))])

```

7.3.3 Login + Authenticated Request Flow



Sequence Diagram 3

7.4 CORS Configuration

app/main.py – current implementation

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins_list,    # env: CORS_ORIGINS,
    allow_methods=["GET", "POST"],
    allow_headers=["Authorization", "Content-Type"],
)

```

X-Staff-ID and X-Staff-Role are intentionally **not** in allow_headers — they are no longer trusted inputs.

7.5 Observability and Audit Hardening

Control	Detail
Correlation ID	CorrelationIDMiddleware sets/propagates X-Correlation-ID; included in every log line and audit row
Structured logging	app/logging_config.py configures python-json-logger; all logs are JSON Lines for ingestion by SIEM
Request log	RequestLoggingMiddleware records method, path, status, duration, correlation_id, actor_id (if auth'd) — no body, no headers
Audit append-only	AuditRepository.update() / .delete() raise OperationNotPermittedError
Sentry	Optional SENTRY_DSN; PII scrubbing rules deny Authorization, Cookie, password keys

7.6 Data Privacy Controls

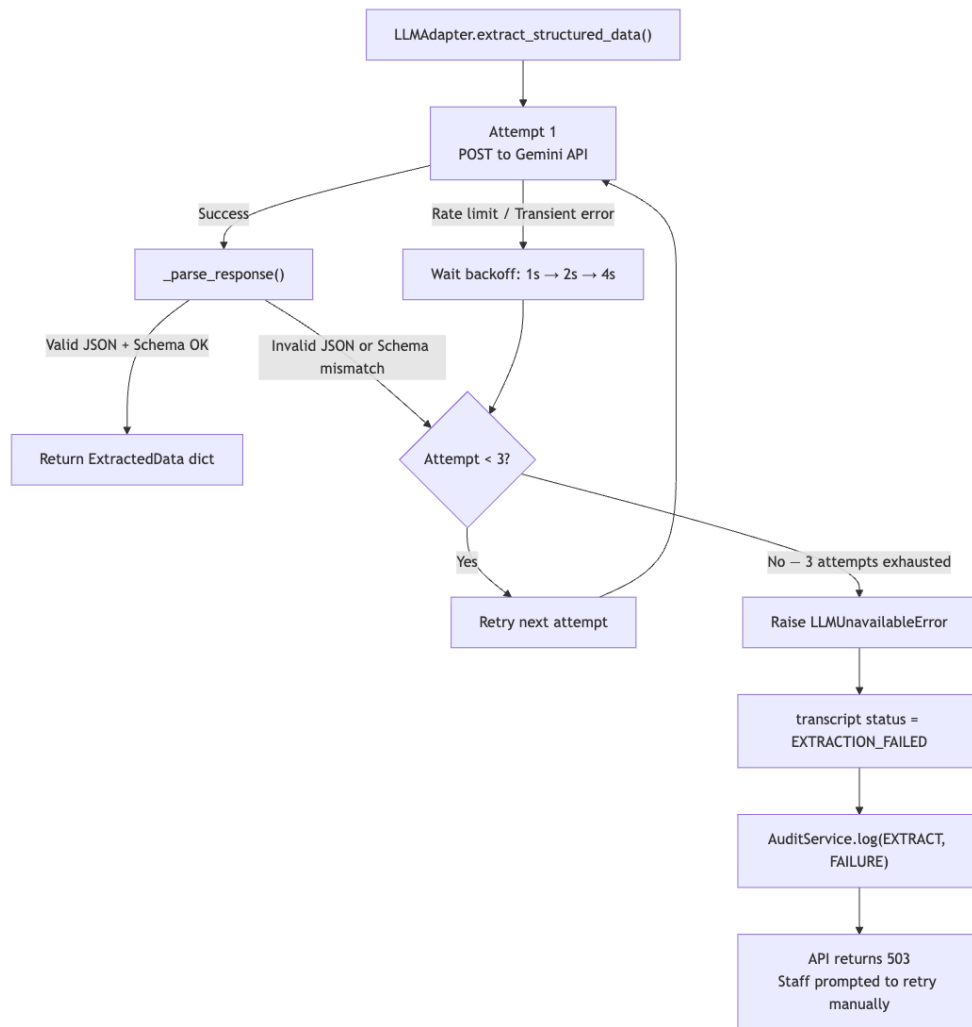
Control	Detail
No live data	Only synthetic / de-identified transcripts used during PoC
Log sanitization	Log messages reference transcript_id only — never student name or PII
LLM transmission	Transcript text sent to Google Gemini over HTTPS only; outbound host allow-listed
Audit detail scrubbing	AuditLog.action_detail JSON omits raw transcript text; references transcript_id for joins
Temp file cleanup	Uploaded files deleted after FILE_RETENTION_HOURS (default 24 h) via background task
Field-level encryption	Reserved for Phase B (DB-010) — not in PoC scope

8. Error Handling & Resilience

8.1 Error Code Reference

Code	HTTP Status	Trigger Condition
INVALID_FILE_TYPE	400	File is not PDF / PNG / JPEG / TIFF
FILE_TOO_LARGE	413	File exceeds 10MB
TRANSCRIPT_NOT_FOUND	404	Transcript ID does not exist
TRANSCRIPT_NOT_PROCESSED	409	Attempting to review a transcript not yet verified
FLAG_ALREADY_REVIEWED	409	A review already exists for this flag
EXTRACTION_FAILED	500	OCR / pdfplumber returned empty text after all attempts
LLM_UNAVAILABLE	503	Gemini API unreachable after 3 retries
LLM_PARSE_ERROR	500	Gemini returned non-JSON or schema-invalid response
OVERRIDE_REASON_REQUIRED	422	Decision is OVERRIDDEN but override_reason is missing
VALIDATION_ERROR	422	Pydantic request body validation failure

8.2 LLM Retry and Resilience Strategy



Flowchart 2

8.3 Global Exception Handler

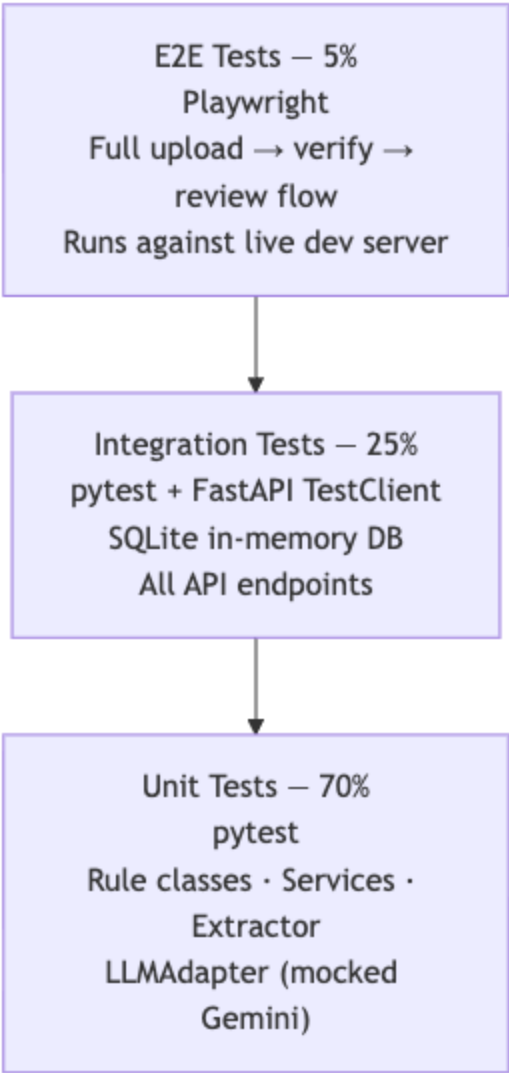
```

@app.exception_handler(MSBONBaseException)
async def msbon_exception_handler(request: Request, exc:
    MSBONBaseException):
    return JSONResponse(
        status_code=exc.http_status,
        content={
            "error": {
                "code": exc.code,
                "message": exc.message,
                "detail": exc.detail,
                "timestamp": datetime.utcnow().isoformat() + "Z",
            }
        },
    )
  
```


All custom exceptions (InvalidFileTypeError, ExtractionFailedError, LLMUnavailableError, etc.) inherit from MSBONBaseException and declare their own http_status, code, and default message.

9. Testing Strategy

9.1 Test Pyramid



Graph 7

9.2 Unit Test Coverage Targets

Module	Key Test Cases
GraduationConfirmationRule	graduation_confirmed=True (pass), False (flag), graduation_date=null (flag)
AccreditationRule	Exact name match, fuzzy match within threshold, no match, inactive program

Module	Key Test Cases
CourseCompletionRule	All required courses present, one missing, credit hours below threshold
FraudIndicatorRule	BSN completed in 2 months (flag), normal timeline (pass), known fraud school (flag)
FormattingConsistencyRule	All fields present (pass), student_name null (flag), mixed date formats (flag)
DocumentExtractor	Valid text PDF, valid image (mocked tesseract), empty PDF (raises)
LLMAdapter	Success on attempt 1, success on attempt 2 (retry), all 3 fail, bad JSON
ReviewService	Valid CONFIRMED, valid OVERRIDDEN with reason, OVERRIDDEN without reason (422)
AuditService	Log creates entry, get_by_transcript returns all entries, no update allowed

9.3 Integration Test Scenarios

Test Scenario	Expected Outcome
Upload valid PDF → process → all rules pass	status=CLEAR, flags=[]
Upload transcript, missing graduation confirmation	status=FLAGGED, GRAD-001 present
Upload transcript from unaccredited school	status=FLAGGED, ACCR-001 present
Submit OVERRIDDEN review without override_reason	422 OVERRIDE_REASON_REQUIRED
Submit valid review → check audit log	Audit entry with action_type=REVIEW_FLAG
Upload file > 10MB	413 FILE_TOO_LARGE
Upload .exe disguised as .pdf	400 INVALID_FILE_TYPE
GET /audit-logs/{id}/export?format=csv	Returns valid CSV binary
GET /health when DB connected	{ "status": "healthy", "database": "connected" }

9.4 Synthetic Test Transcript Inventory

Transcript ID	Description	Expected Flags
TEST-CLEAN-BSN	Valid BSN — all fields complete, ACEN-accredited school	None
TEST-CLEAN-ADN	Valid ADN — all fields complete, CCNE-accredited school	None

Transcript ID	Description	Expected Flags
TEST-NO-GRAD	Transcript without graduation/degree conferral statement	GRAD-001
TEST-NO-DATE	Graduation date field absent	GRAD-002
TEST-PARTIAL	Incomplete program, missing required nursing courses	COUR-001
TEST-FAST-COMPLETE	BSN granted 3 months after enrollment	FRAU-001
TEST-UNKNOWN-SCHOOL	Institution not in accredited_programs table	ACCR-001
TEST-FRAUD-SCHOOL	Institution matches known fraudulent program name	FRAU-003
TEST-SCANNED	Low-quality scanned image transcript	Depends on OCR quality
TEST-FORMAT-BAD	Mixed date formats, missing credit hour fields	FORM-001, FORM-002

9.5 Acceptance Criteria (from Scope Document)

Criterion	Target	How Measured
Flag detection accuracy	≥ 90%	Against synthetic test set
Flag explainability	100%	Every flag has: rule ID + source excerpt + explanation
Audit completeness	100%	Every state change produces an audit_log entry
Override traceability	100%	Every OVERRIDDEN decision has a stored override_reason
No automated decisions	100%	No endpoint approves or denies licensure
Staff usability	Positive feedback	Qualitative review with Dr. Burks, Dr. Smith, Talisha Greenlaw

10. Non-Functional Requirements

Category	Requirement	Target	Notes
Performance	Upload response time	< 2 seconds	File save + DB insert

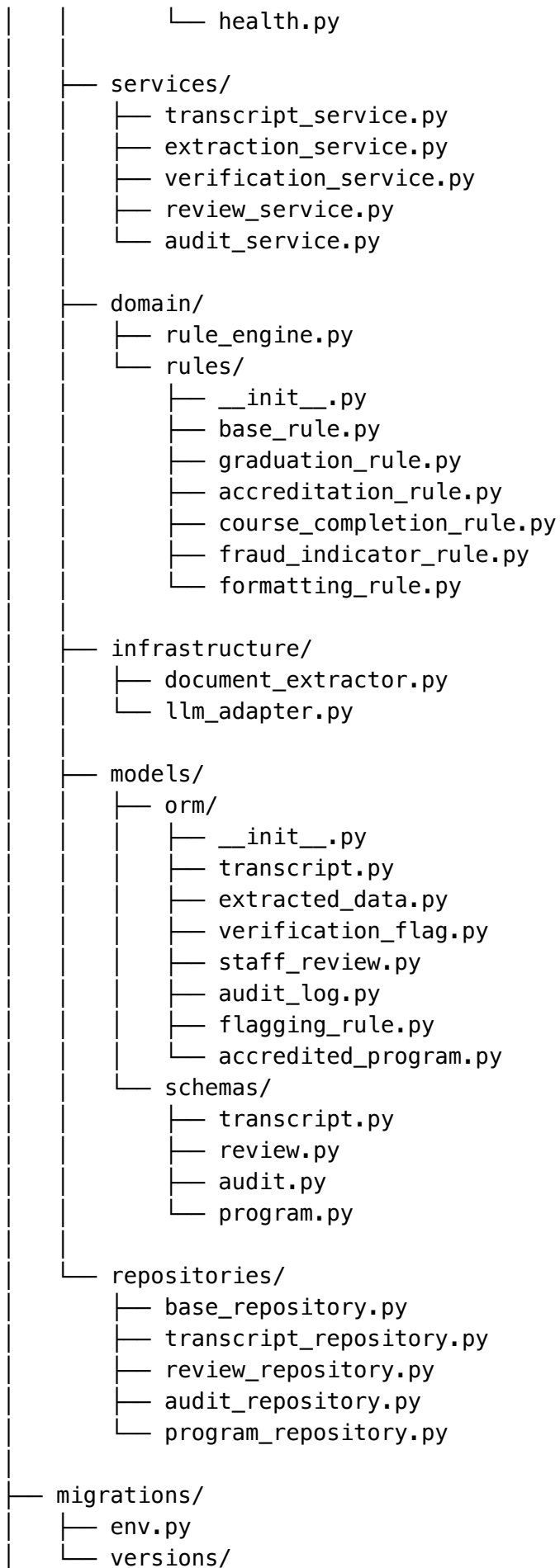
Category	Requirement	Target	Notes
Performance	End-to-end verification	< 30 seconds	OCR + LLM + rules
Performance	Review submit response	< 1 second	DB insert only
Accuracy	Flag detection rate	≥ 90%	Against synthetic test set
Explainability	Flags with full explanation	100%	Scope requirement
Auditability	Actions logged	100%	Zero unlogged state changes
Security	File size limit	10 MB	Enforced at API middleware
Security	Accepted file types	PDF, PNG, JPEG, TIFF	Validated by magic bytes
Privacy	Temp file retention	≤ 24 hours	Configurable via FILE_RETENTION_HOURS
Compatibility	Browser support	Chrome 120+, Firefox 120+, Edge 120+	
Runtime	Python version	3.11+	
Runtime	Node.js version	18+	

11. Appendix

11.1 Project Directory Structure

```
msbon-app/  
├── backend/  
│   ├── app/  
│   │   ├── __init__.py  
│   │   └── main.py  
│   ├── CORS, routers  
│   ├── config.py  
│   ├── .env  
│   ├── database.py  
│   └── session dependency  
│       ├── api/  
│       │   └── v1/  
│       │       ├── __init__.py  
│       │       ├── transcripts.py  
│       │       ├── reviews.py  
│       │       ├── audit.py  
│       │       └── programs.py
```

FastAPI app factory,
pydantic-settings –
SQLAlchemy engine +



```
├── tests/
│   ├── unit/
│   │   ├── test_rules.py
│   │   ├── test_services.py
│   │   ├── test_extractor.py
│   │   └── test_llm_adapter.py
│   ├── integration/
│   │   ├── test_transcripts_api.py
│   │   ├── test_reviews_api.py
│   │   └── test_audit_api.py
│   └── fixtures/
│       ├── synthetic_transcripts/
│       └── conftest.py
├── data/                                # gitignored - SQLite DB file
├── uploads/                            # gitignored - temp transcript
storage
├── logs/                              # gitignored - app log files
├── requirements.txt
├── requirements-dev.txt
├── alembic.ini
├── .env.example
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── UploadPage.tsx
│   │   │   ├── TranscriptListPage.tsx
│   │   │   ├── VerificationPage.tsx
│   │   │   ├── AuditLogPage.tsx
│   │   │   └── ProgramsPage.tsx
│   │   ├── components/
│   │   │   ├── upload/
│   │   │   │   ├── FileDropzone.tsx
│   │   │   │   └── UploadProgressBar.tsx
│   │   │   ├── verification/
│   │   │   │   ├── TranscriptSummary.tsx
│   │   │   │   ├── FlagList.tsx
│   │   │   │   ├── FlagItem.tsx
│   │   │   │   └── RuleExplanation.tsx
│   │   │   ├── review/
│   │   │   │   ├── ReviewForm.tsx
│   │   │   │   ├── DecisionButtons.tsx
│   │   │   │   └── AnnotationInput.tsx
│   │   │   └── audit/
│   │   │       ├── AuditLogTable.tsx
│   │   │       ├── AuditLogFilters.tsx
│   │   │       └── ExportButton.tsx
│   │   └── services/
│   │       ├── transcriptClient.ts
│   │       └── reviewClient.ts
```

```

├── auditClient.ts
├── store/
│   └── uiStore.ts
├── types/
│   └── index.ts
├── App.tsx
├── main.tsx
├── package.json
├── vite.config.ts      # Proxy /api → localhost:8000
├── tsconfig.json
├── docs/
│   └── Software_Design_Document.md
└── README.md

```

11.2 Environment Variables

backend/.env:

```

# Google Gemini
GEMINI_API_KEY=AIza...
GEMINI_MODEL=gemini-2.0-flash

# Database
DATABASE_URL=sqlite:///./data/msbon.db

# File storage
UPLOAD_DIR=./uploads
MAX_FILE_SIZE_MB=10
FILE_RETENTION_HOURS=24

# App
LOG_LEVEL=INFO
CORS_ORIGINS=http://localhost:5173
APP_VERSION=1.0.0-poc

```

frontend/.env:

```
VITE_API_BASE_URL=http://localhost:8000/api/v1
```

11.3 Backend Dependency Manifest

requirements.txt:

```

fastapi==0.110.0
uvicorn[standard]==0.27.0
sqlalchemy==2.0.29
alembic==1.13.1
pydantic==2.6.4
pydantic-settings==2.2.1
google-generativeai==0.5.0

```

```
pdfplumber==0.11.0
pytesseract==0.3.10
Pillow==10.3.0
python-multipart==0.0.9
python-magic==0.4.27
```

requirements-dev.txt:

```
pytest==8.1.1
httpx==0.27.0
pytest-asyncio==0.23.6
faker==24.3.0
```

11.4 Frontend Dependency Manifest

Package	Version	Purpose
react	^18.3.0	UI framework
react-dom	^18.3.0	DOM rendering
react-router-dom	^6.22.3	SPA routing
@tanstack/react-query	^5.28.0	Server state management + caching
zustand	^4.5.2	Client UI state
axios	^1.6.8	HTTP client for API calls
tailwindcss	^3.4.0	Utility-first CSS
typescript	^5.4.0	Type safety
vite	^5.2.0	Build tool + dev server
@vitejs/plugin-react	^4.2.0	React Fast Refresh

11.5 Flagging Rules Seed Data

The following 12 records are inserted into `flagging_rules` at database initialization:

rule_id	name	category	severity	is_active
GRAD-001	Missing graduation confirmation	GRADUATION	HIGH	true
GRAD-002	Graduation date absent	GRADUATION	MEDIUM	true
ACCR-001	Institution not in MS accredited list	ACCREDITATION	HIGH	true
ACCR-002	Accreditation body not recognized	ACCREDITATION	MEDIUM	true
COUR-001	Required nursing	COURSE	HIGH	true

rule_id	name	category	severity	is_active
	courses not present			
COUR-002	Insufficient credit hours	COURSE	MEDIUM	true
FRAU-001	Program completion time below minimum	FRAUD	HIGH	true
FRAU-002	Suspicious grade distribution	FRAUD	MEDIUM	true
FRAU-003	Institution on fraud watch list	FRAUD	HIGH	true
FORM-001	Required transcript fields missing	FORMAT	LOW	true
FORM-002	Inconsistent date formatting	FORMAT	LOW	true
FORM-003	Inconsistent grade formatting	FORMAT	LOW	true